

```
        return self.balance

# Added for debugging
def show(self):
    print('        Name:', self.name)
    print('        Balance:', self.balance)
    print('        Password:', self.password)
```

Listing 4-9: A modified `Account` class that raises exceptions

We start by defining our custom `AbortTransaction` exception ❶ so we can use it in this class and in other code that imports this class.

In the `__init__()` method of the `Account` class, we ensure that the amount provided as the starting balance is valid by calling `validateAmount()` ❷. This method uses a `try/except` block to ensure that the starting amount can successfully be converted to an integer. If the call to `int()` fails, it raises a `ValueError` exception, which is caught in the `except` clause. Rather than just allowing the generic `ValueError` to be returned to the caller, the code of this `except` block ❸ executes a `raise` statement, raising our `AbortTransaction` exception, and includes a more meaningful error message string. If the conversion to an integer succeeds, we perform another test. If the user gave a negative amount, we also raise the `AbortTransaction` exception ❹, but with a different error message string.

The `checkPasswordMatch()` method ❺ is called by methods in the `Bank` object to check if the password supplied by the user matches the password saved in the `Account`. If not, we execute another `raise` statement with the same exception, but we supply a more descriptive error message string.

This allows the code of `deposit()` ❻ and `withdraw()` ❼ to be simplified, because these methods assume that the amount has been validated and the password verified before they are invoked. There is an additional check in `withdraw()` to ensure that the user is not trying to withdraw more money than is in the account; if so, we raise the `AbortTransaction` exception with an appropriate description.

Since there is no code in this class to handle an `AbortTransaction` exception, any time one is raised, control is passed back to the caller. If the caller has no code to handle the exception, then control is passed back to the previous